Examining stack usage

Processors for embedded applications have limited memory and therefore the amount of space available on the stack is also limited. You can use Arm Compiler to determine how much stack space is used by the functions in your application code.

The amount of stack that a function uses depends on factors such as the number and type of arguments to the function, local variables in the function, and the optimizations that the compiler performs. For more information on what the stack is used for, see *Stack use in C and C++* in the *Software Development Guide*.

It is good practice to examine the amount of stack used by the functions in your application. You can then consider rewriting your code to reduce stack usage.

To examine the stack usage in your application, use the linker option `--info=stack`. The following example code shows functions with different numbers of arguments:

```
__attribute__((noinline)) int fact(int n)
{
  int f = 1;
  while (n>0)
      f *= n--;
  return f;
}

int foo (int n)
{
  return fact(n);
}

int foo_mor (int a, int b, int c, int d)
{
  return fact(a);
```

```
int main (void)
{
  return foo(10) + foo_mor(10,11,12,13);
}
```

Copy the code example to `file.c` and compile it using the following command:

```
armclang --target=arm-arm-none-eabi -march=armv8-a -c -g file.c -o fi
```

Compiling with the `-g` option generates the DWARF frame information that `armlink` requires for estimating the stack use. Run `armlink` on the object file using `--info=stack`:

```
armlink file.o --info=stack
```

For the example code, armlink shows the amount of stack used by the various functions. Function `foo_mor` has more arguments than function `foo`, and therefore uses more stack.

```
Stack Usage for fact 0xc bytes.
Stack Usage for foo 0x8 bytes.
Stack Usage for foo_mor 0x10 bytes.
Stack Usage for main 0x8 bytes.
```

You can also examine stack usage using the linker option `--callgraph`:

```
armlink file.o --callgraph -o FileImage.axf
```

This outputs a file called `FileImage.htm` which contains the stack usage information for the various functions in the application.

```
fact (ARM, 84 bytes, Stack size 12 bytes, file.o(.text))

[Stack]

Max Depth = 12
Call Chain = fact

[Called By]
    foo_mor
    foo
```

```
foo (ARM, 36 bytes, Stack size 8 bytes, file.o(.text))

[Stack]

Max Depth = 20
Call Chain = foo >> fact

[Calls]
>>   fact

[Called By]
>>   main
foo_mor (ARM, 76 bytes, Stack size 16 bytes, file.o(.text))

[Stack]

Max Depth = 28
Call Chain = foo_mor >> fact

[Calls]
>>   fact

[Called By]
>>   main
main (ARM, 76 bytes, Stack size 8 bytes, file.o(.text))

[Stack]

Max Depth = 36
Call Chain = main >> foo_mor >> fact

[Calls]
>>   foo_mor
>>   foo

[Called By]
>>   __rt_entry_main (via BLX)
```

See *--info* and *--callgraph* in the *armlink User Guide* for more information on these options.

Rate this page:

Developing on Arm

AI and ML

Android Development

Automotive

Cloud-to-Edge and Networking

Gaming, Graphics and VR

High Performance Computing

Internet of Things

Windows on Arm

Architecture

AMBA

Compression Technologies

CoreSight Architectures

CPU Architecture

GPU Architectures

Instruction Sets

Security Architectures

System Architectures

Products

CPU Processors

Graphics and Multimedia

Physical IP

System IP

SoC Design and Simulation

Software Development Tools

Support

Design Reviews and Checklists

Training

Documentation

Licensing

Downloads

Contact Support

Arm Security Center

Community

Communities

Forums

Blogs

About Arm

Leadership

Security

News

Contact Us

Arm Offices

Cookie Policy Glossary Terms of Use Privacy Policy Accessibility Subscription Center Trademarks

Copyright © 1995-2024 Arm Limited (or its affiliates). All rights reserved.